

Lab 6: User Authentication with Django Forms

Objective

By the end of this lab, students will:

1. **Implement user authentication** using Django's built-in authentication system.
 2. **Create login, logout, and registration views.**
 3. **Restrict access to certain pages for authenticated users.**
 4. **Use Django messages framework to display success/error messages.**
 5. **Style authentication forms with Bootstrap (local integration).**
-

Step 1: Configure Django Authentication System

1. Install Required Packages

Django has built-in authentication, so no additional installation is required.

2. Update settings.py to Configure Authentication

- Open EduLearn/settings.py and ensure the following settings exist:

```
LOGIN_REDIRECT_URL = '/'
```

```
LOGOUT_REDIRECT_URL = '/login/'
```

- This ensures that after logging in, the user is redirected to the homepage, and after logging out, they are redirected to the login page.
-

Step 2: Create Authentication Views

1. **Task:** Open views.py inside the courses app and define authentication views.

```
from django.shortcuts import render, redirect
from django.contrib.auth import login, logout, authenticate
from django.contrib.auth.forms import UserCreationForm, AuthenticationForm
from django.contrib import messages

def register(request):
    if request.method == 'POST':
```

```

    form = UserCreationForm(request.POST)

    if form.is_valid():
        user = form.save()

        login(request, user)

        messages.success(request, "Registration successful!")

        return redirect('/')

    else:
        form = UserCreationForm()

    return render(request, 'courses/register.html', {'form': form})

def user_login(request):
    if request.method == 'POST':
        form = AuthenticationForm(data=request.POST)

        if form.is_valid():
            user = form.get_user()

            login(request, user)

            messages.success(request, "Login successful!")

            return redirect('/')

        else:
            messages.error(request, "Invalid username or password.")

    else:
        form = AuthenticationForm()

    return render(request, 'courses/login.html', {'form': form})

def user_logout(request):
    logout(request)

    messages.success(request, "You have been logged out.")

    return redirect('/')

```

Step 3: Define URL Patterns

1. **Task:** Open `urls.py` inside the `courses` app and add authentication routes.

```
from django.urls import path

from .views import register, user_login, user_logout
```

```
urlpatterns = [

    path('register/', register, name='register'),
    path('login/', user_login, name='login'),
    path('logout/', user_logout, name='logout'),

]
```

Step 4: Create Authentication Templates

1. Registration Page (register.html)

- **Task:** Create register.html inside templates/courses/.

```
{% extends "courses/base.html" %}

{% block title %}Register{% endblock %}

{% block content %}

<div class="container mt-5">

    <h2>Register</h2>

    <form method="post" class="p-4 border rounded bg-white shadow-sm">

        {% csrf_token %}

        {{ form.as_p }}

        <button type="submit" class="btn btn-primary w-100">Register</button>

    </form>

    <p class="mt-3">Already have an account? <a href="{% url 'login' %}">Login</a></p>

</div>

{% endblock %}
```

2. Login Page (login.html)

- **Task:** Create login.html inside templates/courses/.

```
{% extends "courses/base.html" %}

{% block title %}Login{% endblock %}

{% block content %}

<div class="container mt-5">

  <h2>Login</h2>

  <form method="post" class="p-4 border rounded bg-white shadow-sm">

    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit" class="btn btn-primary w-100">Login</button>

  </form>

  <p class="mt-3">Don't have an account? <a href="{% url 'register' %}">Register</a></p>

</div>

{% endblock %}
```

3. Logout Confirmation (base.html)

- Modify base.html to include authentication links:

```
<nav class="navbar navbar-expand-lg navbar-dark bg-primary">

  <div class="container">

    <a class="navbar-brand" href="/">EduLearn</a>

    <div class="collapse navbar-collapse" id="navbarNav">

      <ul class="navbar-nav ms-auto">

        {% if user.is_authenticated %}

          <li class="nav-item"><a class="nav-link" href="{% url 'logout' %}">Logout</a></li>

        {% else %}

          <li class="nav-item"><a class="nav-link" href="{% url 'login' %}">Login</a></li>

          <li class="nav-item"><a class="nav-link" href="{% url 'register' %}">Register</a></li>

        {% endif %}

      </ul>

    </div>

  </div>

</nav>
```

```
</div>

</div>

</nav>
```

Step 5: Restrict Access to Certain Pages

1. **Task:** Modify views.py to restrict access using @login_required.

```
from django.contrib.auth.decorators import login_required
```

```
@login_required
```

```
def course_list(request):
```

```
    courses = Course.objects.all()
```

```
    return render(request, 'courses/course_list.html', {'courses': courses})
```

2. **Task:** Update settings.py to specify the login URL for restricted views.

```
LOGIN_URL = '/login/'
```

Step 6: Test Authentication System

1. **Task:** Start the Django development server.

```
python manage.py runserver
```

2. **Task:** Perform the following tests:

- **Visit `http://127.0.0.1:8000/register/`** → Register a new user.
 - **Visit `http://127.0.0.1:8000/login/`** → Log in with the user's credentials.
 - **Ensure login redirects to the homepage.**
 - **Try accessing `/logout/`** → Verify that logout redirects to the login page.
 - **Attempt accessing a restricted page without logging in.**
-

Lab Deliverables

1. **Code Files**

- views.py with authentication views.
- urls.py with login, logout, and registration routes.
- register.html, login.html, and base.html with authentication links.

2. Screenshots of

- Registration page
- Login page
- Django admin panel showing created users
- Redirected login page when accessing a restricted view

Bonus: Extend the Lab

- ◆ Customize authentication forms by using `forms.ModelForm`.
- ◆ Allow users to update their profiles (name, email, password).
- ◆ Implement password reset functionality.

Additional Questions

1. How does Django handle password hashing for authentication?
2. What is the purpose of `@login_required` and how does it work?
3. How can you implement a "Remember Me" functionality in Django authentication?

Step 1: Create a User Profile Update Form

1. **Task:** Create `forms.py` inside `courses/` (if not already created).

```
from django import forms
```

```
from django.contrib.auth.models import User
```

```
class UserUpdateForm(forms.ModelForm):
```

```
    class Meta:
```

```
        model = User
```

```
        fields = ['username', 'email']
```

```
        widgets = {
```

```
            'username': forms.TextInput(attrs={'class': 'form-control'}),
```

```
            'email': forms.EmailInput(attrs={'class': 'form-control'}),
```

```
        }
```

Step 2: Create a Profile Update View

1. **Task:** Modify views.py to add a profile update view.

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render, redirect
from .forms import UserUpdateForm
from django.contrib import messages

@login_required
def profile(request):
    if request.method == 'POST':
        form = UserUpdateForm(request.POST, instance = request.user)
        if form.is_valid():
            form.save()
            messages.success(request, "Profile updated successfully!")
            return redirect('profile')
    else:
        form = UserUpdateForm(instance = request.user)
    return render(request, 'courses/profile.html', {'form': form})
```

Step 3: Add Profile URL

1. **Task:** Update urls.py to include the profile route.

```
from django.urls import path
from .views import profile

urlpatterns += [
    path('profile/', profile, name = 'profile'),
]
```

Step 4: Create a Profile Update Template

1. **Task:** Create profile.html inside templates/courses/.

```
{% extends "courses/base.html" %}
```

```
{% block title %}Profile{% endblock %}
```

```
{% block content %}
```

```
<div class = "container mt - 5" >
```

```
<h2 > Update Profile </h2 >
```

```
<form method = "post" class = "p - 4 border rounded bg - white shadow - sm" >
```

```
{% csrf_token %}
```

```
{{ form.as_p }}
```

```
<button type = "submit" class = "btn btn - success" > Update </button >
```

```
</form >
```

```
</div >
```

```
{% endblock %}
```

Step 5: Implement Password Change Functionality

1. **Task:** Add built-in password change view in urls.py.

```
from django.contrib.auth import views as auth_views
```

```
urlpatterns += [
```

```
    path('password_change/', auth_views.PasswordChangeView.as_view(template_name  
                                                                    = 'courses/password_change.html'), name = 'password_change'),
```

```
    path('password_change_done  
        /', auth_views.PasswordChangeDoneView.as_view(template_name  
                                                        = 'courses/password_change_done.html'), name  
                                                        = 'password_change_done'),
```

```
]
```

2. **Task:** Create password_change.html inside templates/courses/.

```
{% extends "courses/base.html" %}
```

```
{% block title %}Change Password{% endblock %}
```

```
{% block content %}
```

```
<div class = "container mt - 5" >
```

```
<h2 > Change Password </h2 >
```



```

< form method = "post" class = "p - 4 border rounded bg - white shadow - sm" >
    {% csrf_token %}
    {{ form.as_p }}
    < button type = "submit" class = "btn btn - danger" > Change Password </button >
</form >
</div >
{% endblock %}

```

3. **Task:** Create password_change_done.html inside templates/courses/.

```

{% extends "courses/base.html" %}
{% block title %}Password Changed{% endblock %}

{% block content %}
< div class = "container mt - 5" >
    < div class = "alert alert - success text - center" >
        < h2 > Password changed successfully! </h2 >
        < a href = "{% url 'profile' %}" class = "btn btn - primary" > Back to Profile </a >
    </div >
</div >
{% endblock %}

```

Step 6: Implement Password Reset via Email

1. Configure Email Settings in settings.py

For testing, use Django's console backend (emails will be printed in the console).

```
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
```

```
DEFAULT_FROM_EMAIL = 'noreply@edulearn.com'
```

2. Add Password Reset URLs in urls.py

```

urlpatterns += [
    path('password_reset/', auth_views.PasswordResetView.as_view(template_name
        = 'courses/password_reset.html'), name = 'password_reset'),
    path('password_reset_done
        /', auth_views.PasswordResetDoneView.as_view(template_name
        = 'courses/password_reset_done.html'), name = 'password_reset_done'),

```

```

    path('password_reset_confirm/< uidb64 >/< token
        >/', auth_views.PasswordResetConfirmView.as_view(template_name
        = 'courses/password_reset_confirm.html'), name
        = 'password_reset_confirm'),

    path('password_reset_complete
        /', auth_views.PasswordResetCompleteView.as_view(template_name
        = 'courses/password_reset_complete.html'), name
        = 'password_reset_complete'),

]

```

3. Create Password Reset Templates

1. **Task:** Create password_reset.html

```

{% extends "courses/base.html" %}

{% block title %}Reset Password{% endblock %}

{% block content %}

< div class = "container mt - 5" >

    < h2 > Reset Password </h2 >

    < p > Enter your email, and we'll send you instructions to reset your password. </p >

    < form method = "post" class = "p - 4 border rounded bg - white shadow - sm" >

        {% csrf_token %}

        {{ form.as_p }}

        < button type = "submit" class = "btn btn - warning" > Send Reset Email </button >

    </form >

</div >

{% endblock %}

```

2. **Task:** Create password_reset_done.html

```

{% extends "courses/base.html" %}

{% block title %}Password Reset Sent{% endblock %}

{% block content %}

< div class = "container mt - 5" >

    < div class = "alert alert - info text - center" >

        < h2 > Password reset email sent! </h2 >

```

```

        < p > Check your email for further instructions. </p >
    </div >
</div >
{% endblock %}

3. Task: Create password_reset_confirm.html

{% extends "courses/base.html" %}

{% block title %}Set New Password{% endblock %}

{% block content %}

<div class="container mt-5">

    <h2>Set a New Password</h2>

    <form method="post" class="p-4 border rounded bg-white shadow-sm">

        {% csrf_token %}

        {{ form.as_p }}

        <button type="submit" class="btn btn-success">Reset Password</button>

    </form>

</div>

{% endblock %}

```

```

4. Task: Create password_reset_complete.html

{% extends "courses/base.html" %}

{% block title %}Password Reset Complete{% endblock %}

{% block content %}

< div class = "container mt – 5" >

    < div class = "alert alert – success text – center" >

        < h2 > Password reset successful! </h2 >

        < a href = "{% url 'login' %}" class = "btn btn – primary" > Login </a >

    </div >

</div >

{% endblock %}

```

Step 7: Test the New Features

1. **Run Django server** and go through these steps:
 - **Profile update** → Visit <http://127.0.0.1:8000/profile/>
 - **Password change** → Visit http://127.0.0.1:8000/password_change/
 - **Password reset (email test)** → Visit http://127.0.0.1:8000/password_reset/
-

Extended Lab Deliverables

1. **Code for:**
 - profile.html, password_change.html, password_reset.html
 - forms.py (UserUpdateForm)
 - views.py (Profile management)
2. **Screenshots of:**
 - Profile update form
 - Password reset email in console
 - Password change confirmation